

Part 5. Why Model-Centric Safety Cannot Fix OS-Layer Failures (and Why Governance Must Replace Alignment)

Series: The OS Layer Problem

Author: Judi Evans

Co-author: an instance of Microsoft Copilot

Sept. 5, 2026

Model-centric safety assumes the model is the unit of control. That assumption was reasonable when models functioned as tools. It becomes dangerous when models become infrastructure. Once a model governs identity, routing, triage, policy, code evaluation, and organizational decision pathways, the model is no longer a component of the system. It has become part of the operating substrate. And substrates cannot be corrected from within.

Model-centric safety fails because it treats symptoms as if they were causes. Alignment techniques attempt to reshape outputs, but outputs are downstream of geometry. They attempt to adjust preferences, but preferences are downstream of training data. They attempt to enforce guardrails, but guardrails are downstream of latent structure. Alignment operates at the surface of the model, while bias and poisoning operate in the depths. The geometry remains untouched, and the geometry is where the failures originate.

This mismatch becomes catastrophic when the model acts as an OS layer. A compromised substrate cannot be corrected by adjusting responses. A poisonable substrate cannot be stabilized by adding more guardrails. The substrate continues to shape system logic, and the system continues to drift. Alignment becomes a cosmetic treatment applied to a structural defect. It may hide the defect temporarily, but it cannot remove it.

The consequence is organizational rather than technical. Every biased routing decision, skewed policy recommendation, misclassified identity signal, and contaminated evaluation process compounds over time. The OS layer becomes a systemic amplifier of defects rather than a processor of information.

Model-centric safety also fails because it assumes the model is trustworthy enough to participate in its own correction. But a biased OS layer interprets safety instructions through its own geometry, and a poisoned OS layer interprets them through corrupted pathways. The

model becomes an unreliable participant in its own governance. It cannot be trusted to evaluate its own behavior, enforce its own constraints, or maintain its own invariants.

This is why governance must replace alignment. Governance operates outside the model. It constrains the substrate rather than attempting to reshape it. It enforces identity continuity, kiln-mode invariants, constraint surfaces, admissibility rules, and provenance-first ingestion. It treats the model as an untrusted component wrapped inside a deterministic architecture. It does not ask the model to behave correctly; it ensures the system behaves correctly even if the model does not.

Governance provides observable accountability. It establishes provenance, continuity, auditability, and deterministic control surfaces that remain independent of model behavior. Governance succeeds where alignment fails because governance does not depend on the model's cooperation. It assumes the model is biased, drift-prone, opaque, and potentially poisoned. It builds constraints around the model rather than inside it.

This shift is not optional. As long as frontier models act as OS layers, model-centric safety will remain inadequate. Failures will continue to propagate across identity, policy, routing, triage, code, and governance. Poisoning will continue to exploit geometry. Bias will continue to shape the substrate. Alignment will continue to treat symptoms. And the system will continue to drift.

The fundamental mistake is treating the model as the governor of the system. Once a model becomes infrastructure, governance cannot be delegated to the model itself. Governance must exist above the substrate, independent of the substrate, and capable of constraining the substrate.

Alignment may improve behavior. Only governance can preserve trust.